

Object-Oriented Programming Assignment

Inventory Management System

Group Members

- | | |
|----------------------------------|---------------------|
| 1. Name -Mehul Gupta | Id no:2023A7PS0128U |
| 2. Name -Nikunj Malu | Id no:2023A7PS0177U |
| 3. Name -Isharjot Singh Pasricha | Id no:2023A7PS0147U |
| 4. Name -Aarav Rasquinha | Id no:2023ACPS0317U |
| 5. Name -Sambhram C Kallikote | Id no:2023A7PS0336U |

Submitted To: **Sujala D Shetty**

Date Submitted: **02/12/2024**

Acknowledgement

We would like to express our gratitude to our faculty for allowing us to implement this assignment using the Java programming language. This project enabled us to apply object-oriented programming (OOP) concepts and enhanced our understanding of their practical implementation. It also taught us valuable lessons in teamwork and collaboration.

Table of Contents

1. **Concepts Used**
 - Inheritance
 - GUI (Graphical User Interface)
 - Method Overriding
 - ArrayList
 - Encapsulation
 - User-Defined Exceptions
 - Abstract Classes and Interfaces
2. **Introduction**
3. **Code Overview**
4. **Key Features**
5. **Conclusion**

SOURCE CODE

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.util.ArrayList;
import java.time.LocalDate;

public class InventoryGUI {
```

```

private InventoryManager manager = new InventoryManager();
private ArrayList<Owner> owners = new ArrayList<>();

public InventoryGUI() {
    createMainMenu();
}
Dimension screenSize=Toolkit.getDefaultToolkit().getScreenSize();
int buttonWidth = 400;
int buttonHeight = 100;
int horizontalCenter = screenSize.width / 2 - buttonWidth / 2; // Center horizontally
int verticalGap = 30; // Gap between buttons
Font buttonFont = new Font("Arial",Font.BOLD,24);
Font textFont = new Font("Arial",Font.BOLD,52);
public void createMainMenu() {
    JFrame mainFrame = new JFrame("Inventory Management System");
    mainFrame.setSize(screenSize.width,screenSize.height);
    mainFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    mainFrame.setLayout(null);
    mainFrame.getContentPane().setBackground(Color.WHITE);
    JButton ownerButton = new JButton("Login as Owner");
    JButton customerButton = new JButton("Login as Customer");
    JButton exitButton = new JButton("Exit");

    ownerButton.addActionListener(e -> {
        mainFrame.dispose();
        loginOwnerMenu();
    });

    customerButton.addActionListener(e -> {
        mainFrame.dispose();
        customerMenu();
    });
    int buttonWidth = 400;
    int buttonHeight = 100;
    int horizontalCenter = screenSize.width / 2 - buttonWidth / 2; // Center horizontally
    int verticalGap = 30; // Gap between buttons

    // Set bounds for the buttons (x, y, width, height)
    ownerButton.setBounds(horizontalCenter, screenSize.height / 2 - buttonHeight - verticalGap, buttonWidth,
buttonHeight);
    ownerButton.setFont(buttonFont);
    customerButton.setBounds(horizontalCenter, screenSize.height / 2 + verticalGap, buttonWidth, buttonHeight);
    customerButton.setFont(buttonFont);
    exitButton.setBounds(screenSize.width - 130, 20, 100, 50);
    exitButton.setFont(buttonFont);
    exitButton.addActionListener(e -> System.exit(0));
    mainFrame.add(ownerButton);
    mainFrame.add(customerButton);
    exitButton.setBackground(new Color(255,0,0));
    exitButton.setBorder(null);
    mainFrame.add(exitButton);

    mainFrame.setVisible(true);
}

```

```

private void loginOwnerMenu() {
    JFrame ownerFrame = new JFrame("Owner Login");
    ownerFrame.setSize(screenSize.width, screenSize.height);
    ownerFrame.setLayout(null);
    ownerFrame.getContentPane().setBackground(Color.WHITE);
    JLabel usernameLabel = new JLabel("Username:");
    JTextField usernameField = new JTextField();
    JLabel passwordLabel = new JLabel("Password:");
    JPasswordField passwordField = new JPasswordField();
    JButton loginButton = new JButton("Login");
    JButton createButton = new JButton("Create New Owner");
    JButton backButton = new JButton("Back");

    loginButton.addActionListener(e -> {
        String username = usernameField.getText();
        String password = new String(passwordField.getPassword());

        boolean found = false;
        for (Owner owner : owners) {
            if (owner.getUsername().equals(username) && owner.getPassword().equals(password)) {
                found = true;
                JOptionPane.showMessageDialog(ownerFrame, "Login Successful!");
                ownerFrame.dispose();
                ownerMenu();
                break;
            }
        }
        if (!found) {
            JOptionPane.showMessageDialog(ownerFrame, "Invalid Credentials!");
        }
    });

    createButton.addActionListener(e -> {
        String username = JOptionPane.showInputDialog("Enter Username:");
        String password = JOptionPane.showInputDialog("Enter Password:");
        if (username != null && password != null) {
            owners.add(new Owner(username, password));
            JOptionPane.showMessageDialog(ownerFrame, "Owner Created Successfully!");
        }
    });

    backButton.addActionListener(e -> {
        ownerFrame.dispose();
        createMainMenu();
    });

    usernameLabel.setBounds(screenSize.width-150, screenSize.height-80, 300, 100);
    usernameLabel.setFont(buttonFont);
    ownerFrame.add(usernameLabel);
    usernameField.setBounds(screenSize.width-130, screenSize.height-80, 500, 100);
    usernameField.setFont(buttonFont);
    ownerFrame.add(usernameField);
    passwordLabel.setBounds(screenSize.width-150, screenSize.height-60, 300, 100);
    passwordLabel.setFont(buttonFont);

```

```

ownerFrame.add(passwordLabel);
passwordField.setBounds(screenSize.width-1300,screenSize.height-600,500,100);
passwordField.setFont(buttonFont);
ownerFrame.add(passwordField);
loginButton.setBounds(screenSize.width-1300,screenSize.height-400,300,100);
loginButton.setFont(buttonFont);
ownerFrame.add(loginButton);
createButton.setBounds(screenSize.width-900,screenSize.height-400,300,100);
createButton.setFont(buttonFont);
ownerFrame.add(createButton);
backButton.setBounds(screenSize.width - 130, 20, 100, 50);
backButton.setFont(buttonFont);
backButton.setBackground(new Color(255,0,0));
backButton.setBorder(null);
ownerFrame.add(backButton);

ownerFrame.setVisible(true);
}

private void ownerMenu() {
    JFrame ownerMenuFrame = new JFrame("Owner Menu");
    ownerMenuFrame.setSize(screenSize.width,screenSize.height);
    ownerMenuFrame.setLayout(null);
    ownerMenuFrame.getContentPane().setBackground(Color.WHITE);
    JButton addItemButton = new JButton("Add Item");
    JButton viewItemsButton = new JButton("View Items");
    JButton removeItemButton = new JButton("Remove Item");
    JButton subtractItemButton = new JButton("Reduce Quantity");
    JButton revenueButton = new JButton("View Revenue");
    JButton lowStockButton = new JButton("Check Low Stock");
    JButton expiredItemsButton = new JButton("Check Expired Items");
    JButton logoutButton = new JButton("Logout");
    JLabel ownermenuLabel = new JLabel("OWNER'S MENU");

    addItemButton.addActionListener(e -> addItem());
    viewItemsButton.addActionListener(e -> viewItems());
    removeItemButton.addActionListener(e -> removeItem());
    subtractItemButton.addActionListener(e -> subtractItem());
    revenueButton.addActionListener(e -> JOptionPane.showMessageDialog(ownerMenuFrame,
        "Total Revenue: $" + manager.getTotalRevenue()));
    lowStockButton.addActionListener(e -> {
String thresholdStr = JOptionPane.showInputDialog("Enter stock threshold:");
if (thresholdStr != null) {
    int threshold = Integer.parseInt(thresholdStr);
    ArrayList<InventoryItem> lowStockItems = manager.getLowStockItems(threshold);
    if (lowStockItems.isEmpty()) {
        JOptionPane.showMessageDialog(ownerMenuFrame, "No items are below the threshold.");
    } else {
        StringBuilder items = new StringBuilder("Low Stock Items:\n");
        for (InventoryItem item : lowStockItems) {
            items.append(item.getDetails()).append("\n");
        }
        JOptionPane.showMessageDialog(ownerMenuFrame, items.toString());
    }
}
}
}

```

```

    }
});
expiredItemsButton.addActionListener(e -> {
    ArrayList<PerishableItem> expiredItems = manager.getExpiredItems();
    if (expiredItems.isEmpty()) {
        JOptionPane.showMessageDialog(ownerMenuFrame, "No expired items.");
    } else {
        StringBuilder items = new StringBuilder("Expired Items:\n");
        for (PerishableItem item : expiredItems) {
            items.append(item.getDetails()).append("\n");
        }
        JOptionPane.showMessageDialog(ownerMenuFrame, items.toString(), "Expired Items",
JOptionPane.WARNING_MESSAGE);
    }
});

logoutButton.addActionListener(e -> {
    ownerMenuFrame.dispose();
    createMainMenu();
});
ownerMenuLabel.setBounds(screenSize.width-1200,40,800,100);
ownerMenuLabel.setFont(textFont);
ownerMenuFrame.add(ownerMenuLabel);
addItemButton.setBounds(screenSize.width-1900,screenSize.height-800,280,100);
addItemButton.setFont(buttonFont);
ownerMenuFrame.add(addItemButton);
viewItemsButton.setBounds(screenSize.width-1900,screenSize.height-650,280,100);
viewItemsButton.setFont(buttonFont);
ownerMenuFrame.add(viewItemsButton);
removeItemButton.setBounds(screenSize.width-1900,screenSize.height-500,280,100);
removeItemButton.setFont(buttonFont);
ownerMenuFrame.add(removeItemButton);
subtractItemButton.setBounds(screenSize.width-1570,screenSize.height-800,280,100);
subtractItemButton.setFont(buttonFont);
ownerMenuFrame.add(subtractItemButton);
revenueButton.setBounds(screenSize.width-1900,screenSize.height-350,280,100);
revenueButton.setFont(buttonFont);
ownerMenuFrame.add(revenueButton);
lowStockButton.setBounds(screenSize.width-1570,screenSize.height-650,280,100);
lowStockButton.setFont(buttonFont);
ownerMenuFrame.add(lowStockButton);
expiredItemsButton.setBounds(screenSize.width-1570,screenSize.height-500,280,100);
expiredItemsButton.setFont(buttonFont);
ownerMenuFrame.add(expiredItemsButton);
logoutButton.setBounds(screenSize.width - 200, 20, 150, 50);
logoutButton.setFont(buttonFont);
logoutButton.setBackground(new Color(255,0,0));
logoutButton.setBorder(null);
ownerMenuFrame.add(logoutButton);

ownerMenuFrame.setVisible(true);

}

```

```

private void addItem() {
    JTextField idField = new JTextField();
    JTextField nameField = new JTextField();
    JTextField quantityField = new JTextField();
    JTextField priceField = new JTextField();
    JCheckBox perishableBox = new JCheckBox("Perishable?");
    JTextField expirationField = new JTextField();

    Object[] fields = {
        "Item ID:", idField,
        "Name:", nameField,
        "Quantity:", quantityField,
        "Price:", priceField,
        perishableBox,
        "Expiration Date (YYYY-MM-DD, if Perishable):", expirationField
    };

    int option = JOptionPane.showConfirmDialog(null, fields, "Add Item", JOptionPane.OK_CANCEL_OPTION);
    if (option == JOptionPane.OK_OPTION) {
        int id = Integer.parseInt(idField.getText());
        String name = nameField.getText();
        int quantity = Integer.parseInt(quantityField.getText());
        double price = Double.parseDouble(priceField.getText());

        if (perishableBox.isSelected()) {
            LocalDate expirationDate = LocalDate.parse(expirationField.getText());
            manager.addItem(new PerishableItem(id, name, quantity, price, expirationDate));
        } else {
            manager.addItem(new NonPerishableItem(id, name, quantity, price));
        }
    }
}

private void viewItems() {
    ArrayList<InventoryItem> inventory = manager.getInventory();
    if (inventory.isEmpty()) {
        JOptionPane.showMessageDialog(null, "Inventory is empty.");
    } else {
        StringBuilder items = new StringBuilder();
        for (InventoryItem item : inventory) {
            if (item.getQuantity() < 5) { // Example threshold
                items.append("LOW STOCK ");
            }
            items.append(item.getDetails()).append("\n");
        }
        JOptionPane.showMessageDialog(null, items.toString(), "Inventory Items",
JOptionPane.INFORMATION_MESSAGE);
    }
}

```

```

private void removeItem() {
    String idStr = JOptionPane.showInputDialog("Enter ID of item to remove:");
    if (idStr != null) {
        int id = Integer.parseInt(idStr);
        manager.removeItem(id);
    }
}

private void subtractItem() {
    String idStr = JOptionPane.showInputDialog("Enter ID of item to subtract from:");
    if (idStr != null) {
        int id = Integer.parseInt(idStr);
        String quantityStr = JOptionPane.showInputDialog("Enter quantity to subtract:");
        if (quantityStr != null) {
            int quantity = Integer.parseInt(quantityStr);
            boolean itemFound = false;

            for (InventoryItem item : manager.getInventory()) {
                if (item.getId() == id) {
                    itemFound = true;
                    int currentQuantity = item.getQuantity();
                    if (currentQuantity >= quantity) {
                        // Update quantity
                        item.setQuantity(currentQuantity - quantity);
                        JOptionPane.showMessageDialog(null, "Quantity updated successfully!");
                    } else {
                        JOptionPane.showMessageDialog(null, "Not enough stock to subtract!");
                    }
                    break;
                }
            }

            if (!itemFound) {
                JOptionPane.showMessageDialog(null, "Item with ID " + id + " not found.");
            }
        }
    }
}
}

```

```

private void customerMenu() {
    JFrame customerMenuFrame = new JFrame("Customer Menu");
    customerMenuFrame.setSize(screenSize.width, screenSize.height);
    customerMenuFrame.setLayout(null);
    customerMenuFrame.getContentPane().setBackground(Color.WHITE);
    JButton searchButton = new JButton("Search Item");
    JButton buyButton = new JButton("Buy Item");
    JButton logoutButton = new JButton("Logout");

    searchButton.addActionListener(e -> {
        String name = JOptionPane.showInputDialog("Enter item name to search:");
        if (name != null) {
            boolean found = false;
            for (InventoryItem item : manager.getInventory()) {

```

```

        if (item.getName().equalsIgnoreCase(name)) {
            found = true;
            String status = item.getQuantity() > 0 ? "Available" : "Out of Stock";
            JOptionPane.showMessageDialog(customerMenuFrame, "Item: " + name + "\nPrice: $"
                + item.getPrice() + "\nStatus: " + status);
            break;
        }
    }
    if (!found) {
        JOptionPane.showMessageDialog(customerMenuFrame, "Item not found.");
    }
}
});

buyButton.addActionListener(e -> {
    String name = JOptionPane.showInputDialog("Enter item name to buy:");
    if (name != null) {
        String quantityStr = JOptionPane.showInputDialog("Enter quantity to buy:");
        if (quantityStr != null) {
            int quantity = Integer.parseInt(quantityStr);
            manager.buyItem(name, quantity);
        }
    }
});

logoutButton.addActionListener(e -> {
    customerMenuFrame.dispose();
    createMainMenu();
});

searchButton.setBounds(horizontalCenter, screenSize.height / 2 - buttonHeight - verticalGap, buttonWidth,
buttonHeight);
searchButton.setFont(buttonFont);
buyButton.setBounds(horizontalCenter, screenSize.height / 2 + verticalGap, buttonWidth, buttonHeight);
buyButton.setFont(buttonFont);
customerMenuFrame.add(searchButton);
customerMenuFrame.add(buyButton);
logoutButton.setBounds(screenSize.width - 200, 20, 150, 50);
logoutButton.setFont(buttonFont);
logoutButton.setBackground(new Color(255,0,0));
logoutButton.setBorder(null);
customerMenuFrame.add(logoutButton);

customerMenuFrame.setVisible(true);
}

public static void main(String[] args) {
    new InventoryGUI();
}
}
import java.util.ArrayList;
import java.util.Scanner;
class InventoryManager {
    private ArrayList<InventoryItem> inventory;
    private double totalRevenue;

```

```

public InventoryManager() {
    inventory = new ArrayList<>();
    totalRevenue = 0.0;
}

public void addItem(InventoryItem item) {
    // Check if the item already exists in the inventory
    for (InventoryItem existingItem : inventory) {
        if (existingItem.getId() == item.getId()) {
            // If the item exists, update the quantity
            existingItem.setQuantity(existingItem.getQuantity() + item.getQuantity());
            System.out.println("Item quantity updated successfully!");
            return;
        }
    }
    // If the item does not exist, add the new item to the inventory
    inventory.add(item);
    System.out.println("Item added successfully!");
}

public ArrayList<InventoryItem> getInventory() {
    return inventory;
}

public void viewItems() {
    if (inventory.isEmpty()) {
        System.out.println("Inventory is empty.");
    } else {
        for (InventoryItem item : inventory) {
            System.out.println(item.getDetails());
        }
    }
}

public void removeItem(int id) {
    inventory.removeIf(item -> item.getId() == id);
    System.out.println("Item removed successfully!");
}

public void buyItem(String name, int quantity) {
    for (InventoryItem item : inventory) {
        if (item.getName().equals(name.toLowerCase())) {
            if (item.purchase(quantity)) {
                totalRevenue += item.getPrice() * quantity;
                System.out.println("Purchase successful! You bought " + quantity + " of " + name + ".");
            }
            return;
        }
    }
    System.out.println("Item not found.");
}

public double getTotalRevenue() {

```

```

        return totalRevenue;
    }
    public ArrayList<InventoryItem> getLowStockItems(int threshold) {
        ArrayList<InventoryItem> lowStockItems = new ArrayList<>();
        for (InventoryItem item : inventory) {
            if (item.getQuantity() < threshold) {
                lowStockItems.add(item);
            }
        }
        return lowStockItems;
    }
    public ArrayList<PerishableItem> getExpiredItems() {
        ArrayList<PerishableItem> expiredItems = new ArrayList<>();
        for (InventoryItem item : inventory) {
            if (item instanceof PerishableItem) {
                PerishableItem perishable = (PerishableItem) item;
                if (perishable.isExpired()) {
                    expiredItems.add(perishable);
                }
            }
        }
        return expiredItems;
    }
}

class Owner {
    private String username;
    private String password;

    public Owner(String username, String password) {
        this.username = username;
        this.password = password;
    }

    public String getUsername() { return username; }
    public String getPassword() { return password; }
}

import java.time.LocalDate;

class PerishableItem extends InventoryItem {
    private LocalDate expirationDate;

    public PerishableItem(int id, String name, int quantity, double price, LocalDate expirationDate) {
        super(id, name, quantity, price);
        this.expirationDate = expirationDate;
    }

    public LocalDate getExpirationDate() { return expirationDate; }

    @Override
    public String getDetails() {
        return super.getName() + " - Quantity: " + super.getQuantity() +
            ", Price: $" + super.getPrice() + ", Expiration Date: " + expirationDate;
    }
}

```

```

        public boolean isExpired() {
            return LocalDate.now().isAfter(expirationDate);
        }
    }

    class NonPerishableItem extends InventoryItem {
        public NonPerishableItem(int id, String name, int quantity, double price) {
            super(id, name, quantity, price);
        }

        @Override
        public String getDetails() {
            return super.getName() + " - Quantity: " + super.getQuantity() +
                ", Price: $" + super.getPrice();
        }
    }

    abstract class InventoryItem {
        private int id;
        private String name;
        private int quantity;
        private double price;

        public InventoryItem(int id, String name, int quantity, double price) {
            this.id = id;
            this.name = name.toLowerCase(); // Store name in lowercase
            this.quantity = quantity;
            this.price = price;
        }

        public int getId() { return id; }
        public String getName() { return name; }
        public int getQuantity() { return quantity; }
        public double getPrice() { return price; }

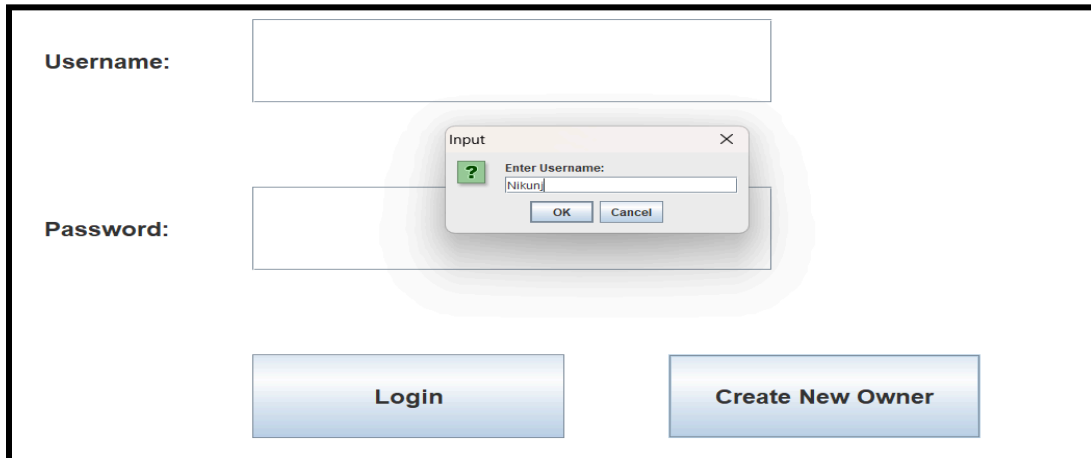
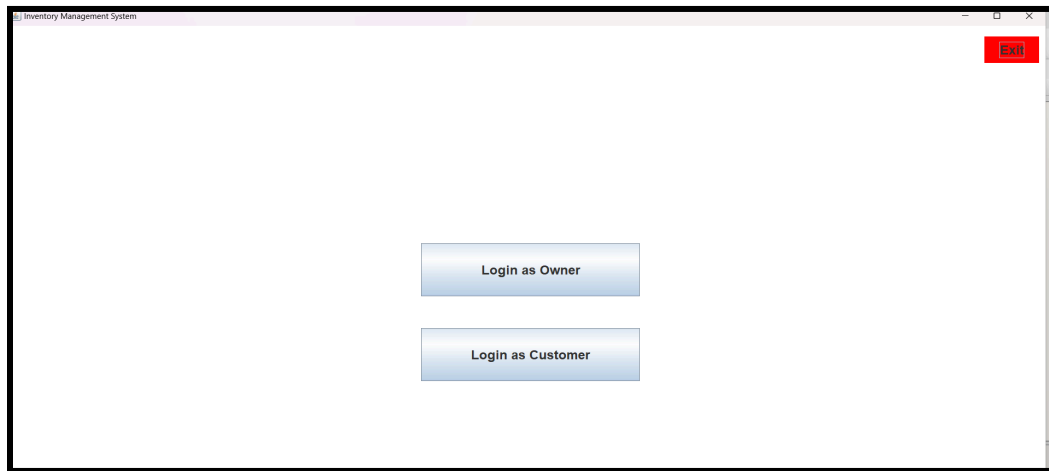
        public void setQuantity(int quantity) { this.quantity = quantity; }

        public boolean purchase(int quantity) {
            if (this.quantity >= quantity) {
                this.quantity -= quantity;
                return true;
            } else {
                System.out.println("Not enough stock for item: " + name);
                return false;
            }
        }

        public abstract String getDetails();
    }

```

Output Screenshot



Input

?

Enter Password:

1234

OK

Cancel

Username:

Nikunj

Password:

....

Message

i

Login Successful!

OK

Login

Create New Owner

OWNER'S MENU

Logout

Add Item

Reduce Quantity

View Items

Check Low Stock

Remove Item

Check Expired Items

View Revenue

OWNER'S MENU

Logout

Add Item

Reduce Quantity

View Items

Check Low Stock

Remove Item

Check Expired Items

View Revenue

Add Item

?

Item ID:
100

Name:
Maggi

Quantity:
40

Price:
5

☒ Perishable?
Expiration Date (YYYY-MM-DD, if Perishable):
2024-12-25

OK

Cancel

Inventory Items

i

maggi - Quantity: 40, Price: \$5.0, Expiration Date: 2024-12-25

OK

Input

?

Enter ID of item to subtract from:

100

OK

Cancel

Input

?

Enter quantity to subtract:

10

OK

Cancel

Message

i

Quantity updated successfully!

OK

Inventory Items

i

maggi - Quantity: 30, Price: \$5.0, Expiration Date: 2024-12-25

OK

Input

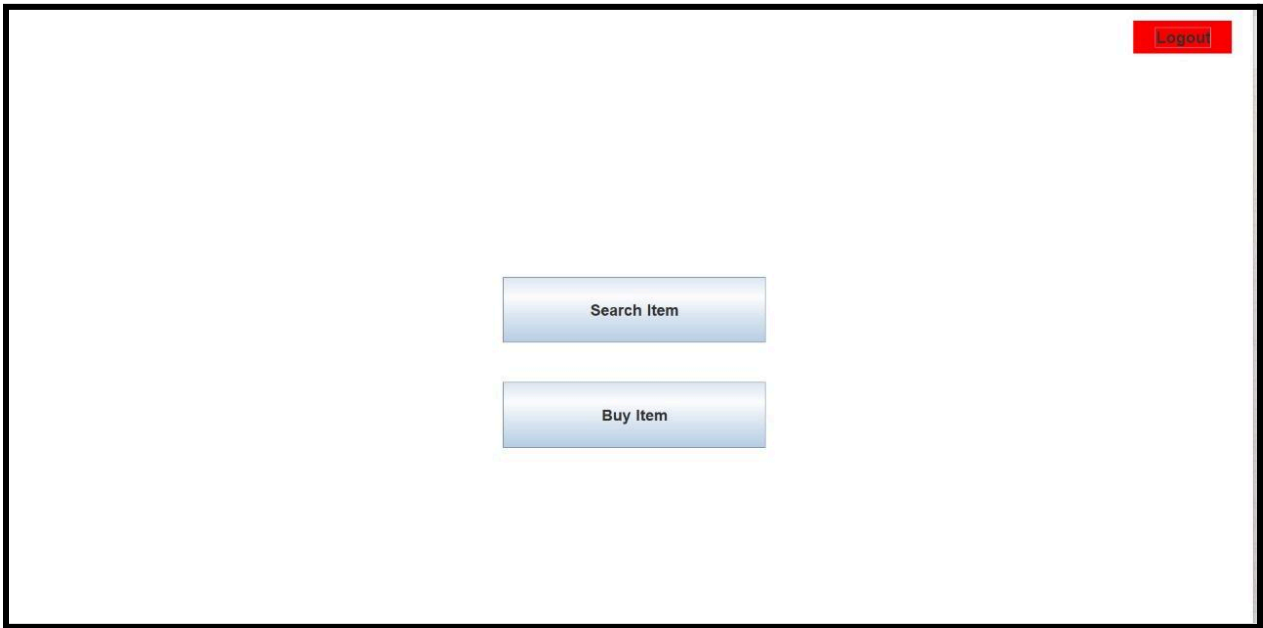
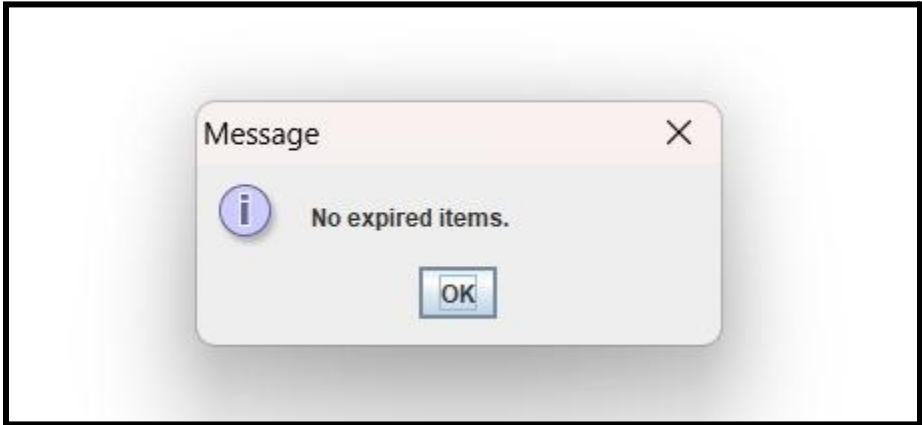
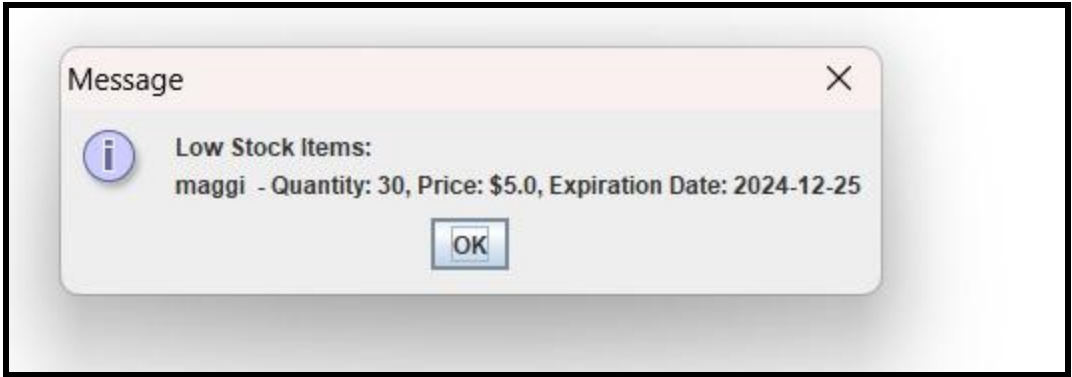
?

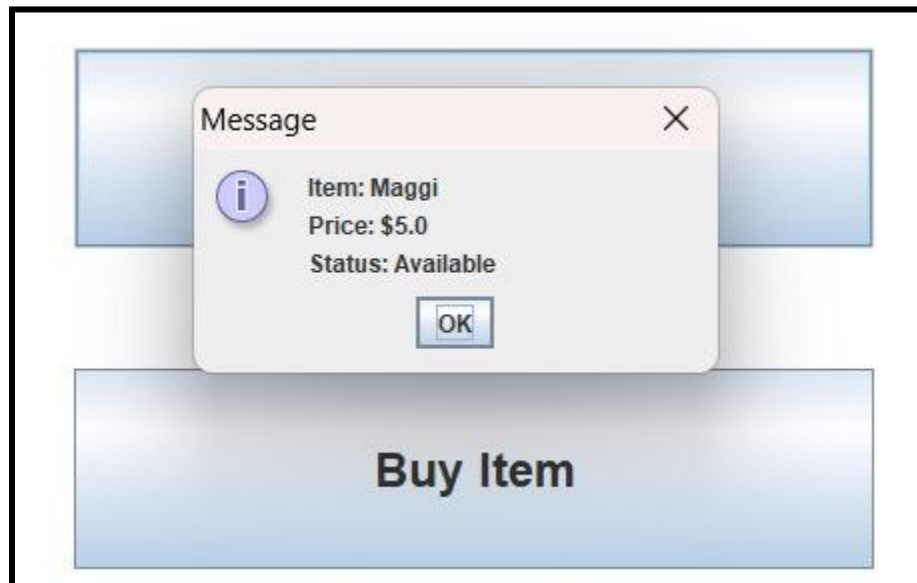
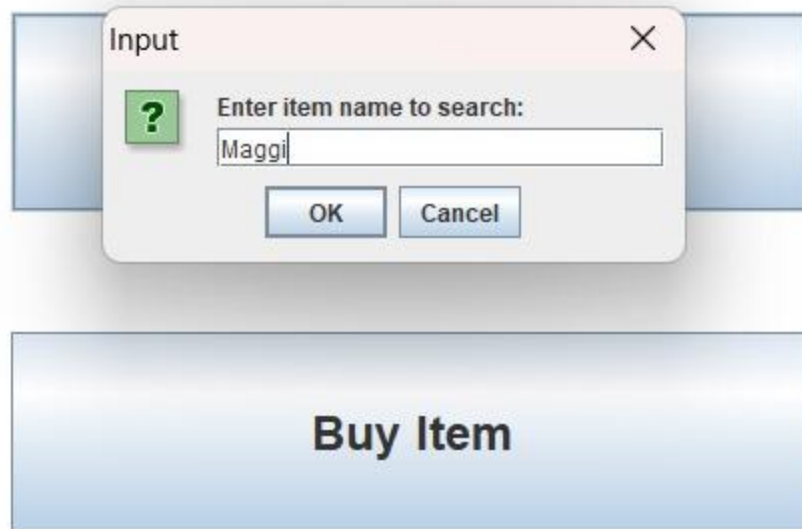
Enter stock threshold:

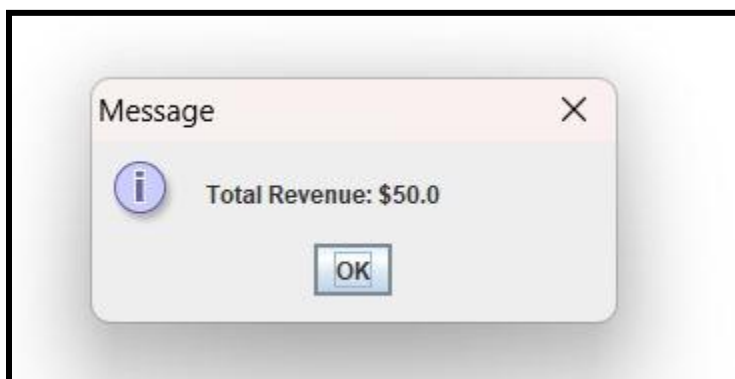
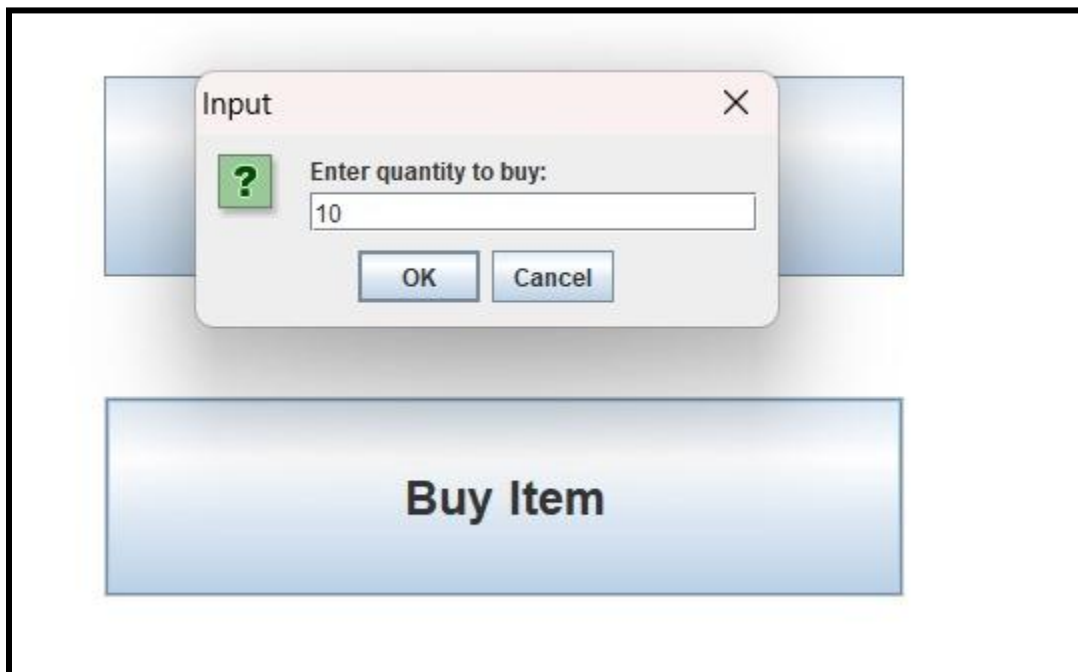
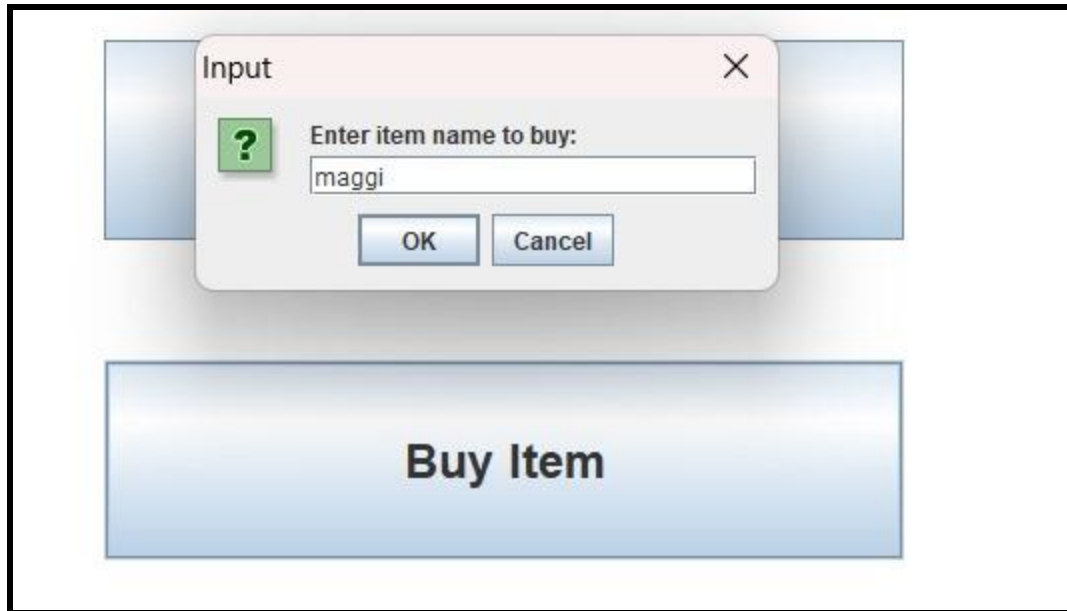
40

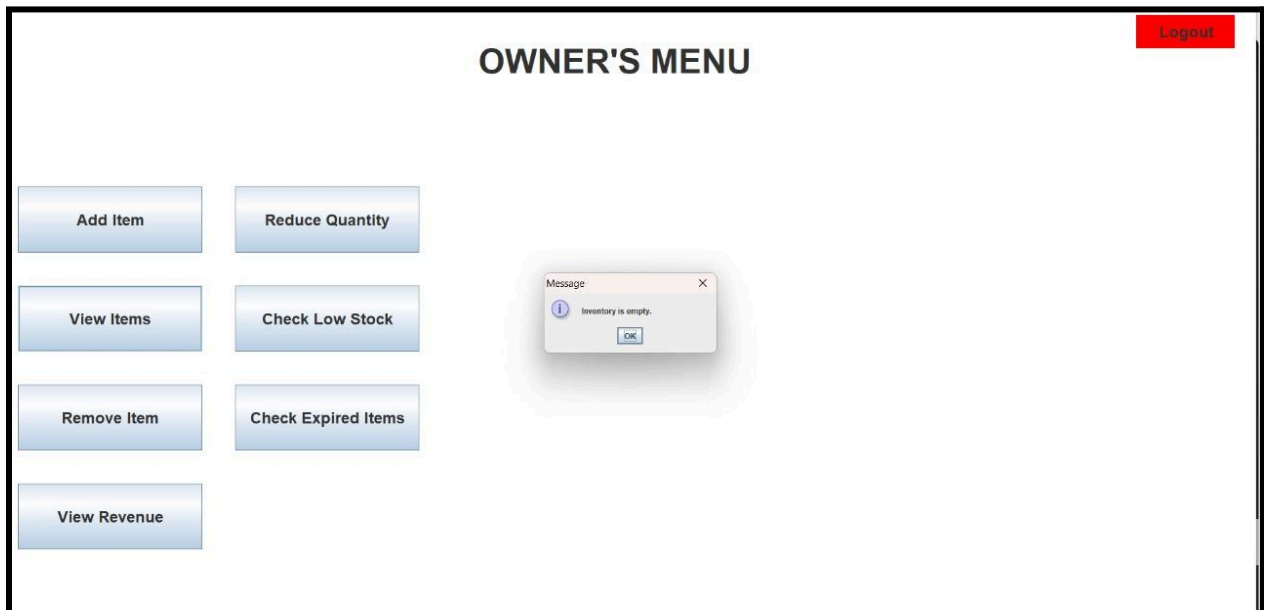
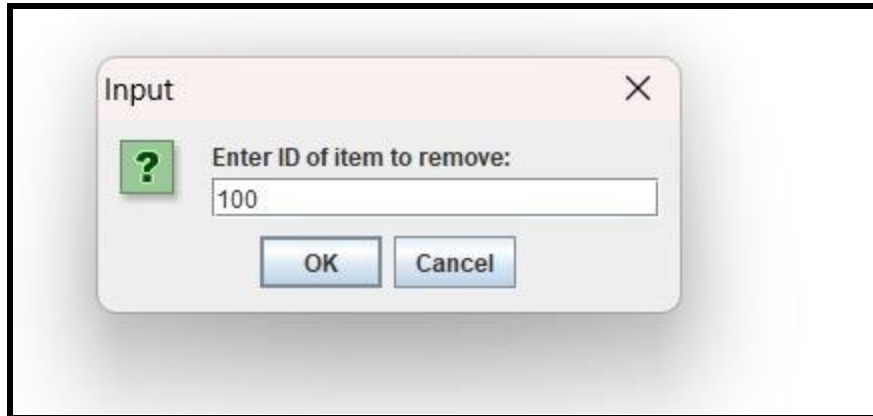
OK

Cancel









Concepts Used

Inheritance

The program demonstrates inheritance, where subclasses such as `PerishableItem` and `NonPerishableItem` inherit properties and methods from the parent class `InventoryItem`. This allows code reuse and better structure.

GUI (Graphical User Interface)

The GUI provides an intuitive interface for users to interact with the application. It includes buttons, text fields, and labels for functionalities such as login, adding items, and viewing inventory.

Method Overriding

The `getDetails()` method is overridden in subclasses like `PerishableItem` and `NonPerishableItem` to provide specific details about each type of item.

ArrayList

`ArrayLists` are used extensively to store and manage collections, such as inventory and user data. This provides dynamic resizing and easy manipulation of data.

Encapsulation

Encapsulation is achieved by using private fields and public getter and setter methods to restrict direct access to the internal state of objects.

User-Defined Exceptions

Custom exceptions, such as `InvalidChoice`, handle specific error scenarios, improving user feedback and robustness of the program.

Abstract Classes and Interfaces

Abstract classes such as `InventoryItem` provide a blueprint for specific types of items, ensuring consistency and enforceable structure.

Introduction

This project simulates an **Inventory Management System** designed to manage store operations. The program consists of two user roles:

1. **Owner:** Manages inventory by adding, removing, or editing items.
2. **Customer:** Browses items, searches, and makes purchases.

The program employs a user-friendly GUI and integrates core OOP concepts to enhance functionality and scalability.

Code Overview

Class Structure

1. **InventoryItem** (Abstract Class)
 - Fields: `id`, `name`, `quantity`, `price`
 - Methods: `getDetails()` (abstract), `purchase()`, `setQuantity()`
2. **PerishableItem** (Extends InventoryItem)
 - Adds `expirationDate` and methods to check expiry.
3. **NonPerishableItem** (Extends InventoryItem)
 - Focuses on general stock items without expiry.
4. **InventoryManager**
 - Handles item addition, removal, stock checking, and purchase processing.
5. **InventoryGUI**
 - Implements the main GUI functionality for interaction.
6. **Owner and Customer Menus**
 - Role-specific options.

Features

- **Login System:** Secure access for owners and customers.
 - **Inventory Management:** Owners can add, remove, and modify inventory.
 - **Stock Check:** Customers can browse and purchase items with real-time stock updates.
 - **Revenue Tracking:** Owners can view revenue generated from purchases.
-

Key Features

Adding Items

Owners can add perishable and non-perishable items to the inventory. The GUI prompts for details such as name, price, quantity, and expiry (if applicable).

Low Stock Alerts

The system alerts owners when stock levels fall below a threshold.

Purchase Flow

Customers can search for items, add them to the cart, and proceed to checkout. Transactions are simulated with a secure credit card processing dialog.

Expiry Checks

The system tracks perishable items and alerts the owner about expired stock.

Conclusion

This project demonstrates the effective use of Object-Oriented Programming principles in creating a functional inventory management system. Key takeaways include:

- Mastery of OOP concepts like inheritance, encapsulation, and polymorphism.
- Implementation of a GUI for enhanced user experience.
- Application of ArrayLists for dynamic data handling.
- Development of robust exception handling for better user feedback.

This assignment not only strengthened our Java skills but also prepared us for real-world application development challenges.